

Design and Implementation of an Arduino-Based Smart Line-Following and Obstacle-Avoiding Vehicle

Team E1 Members

Baris Koluacik

Md Mahbubur Rahman

Md Naimur Rahaman Sakib

Yongyu Xu

CONTENT



PART 1 3D design



PART 2 Objectives & Approach



PART 3 Simulation



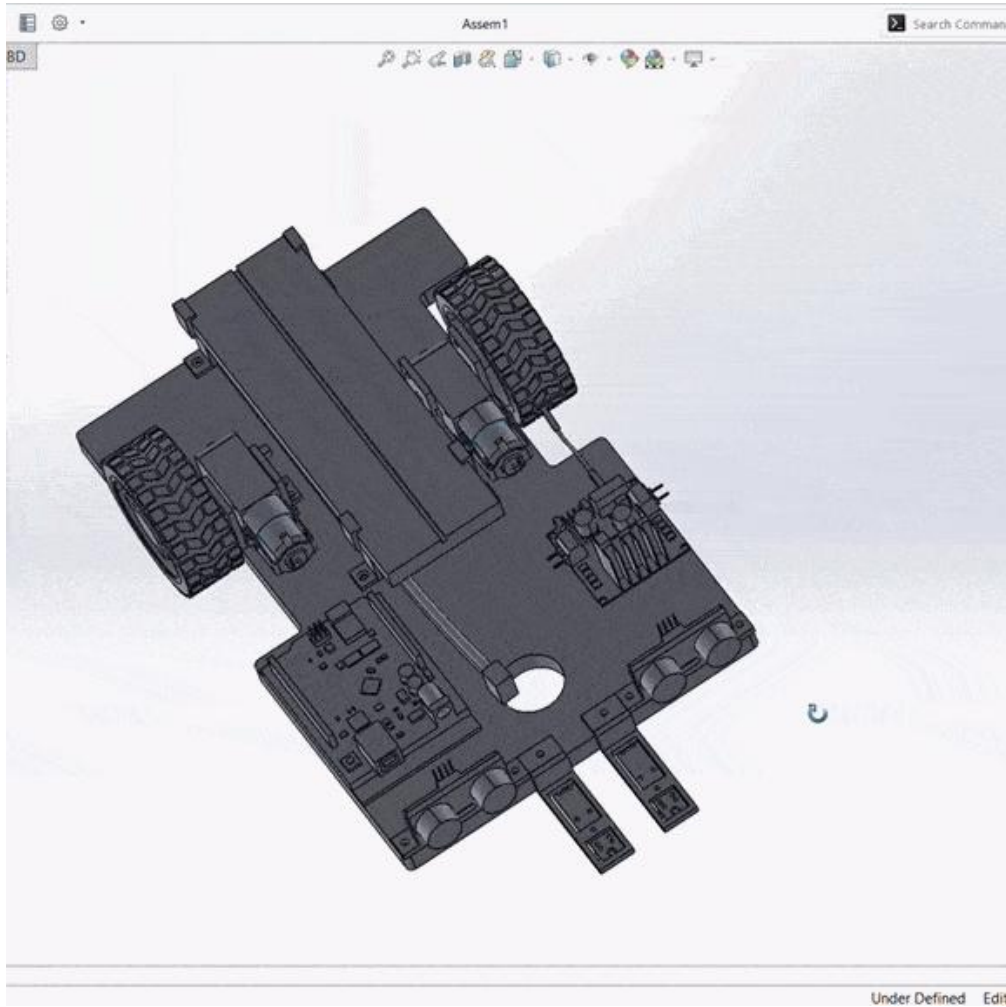
PART 4 Issues and Solutions



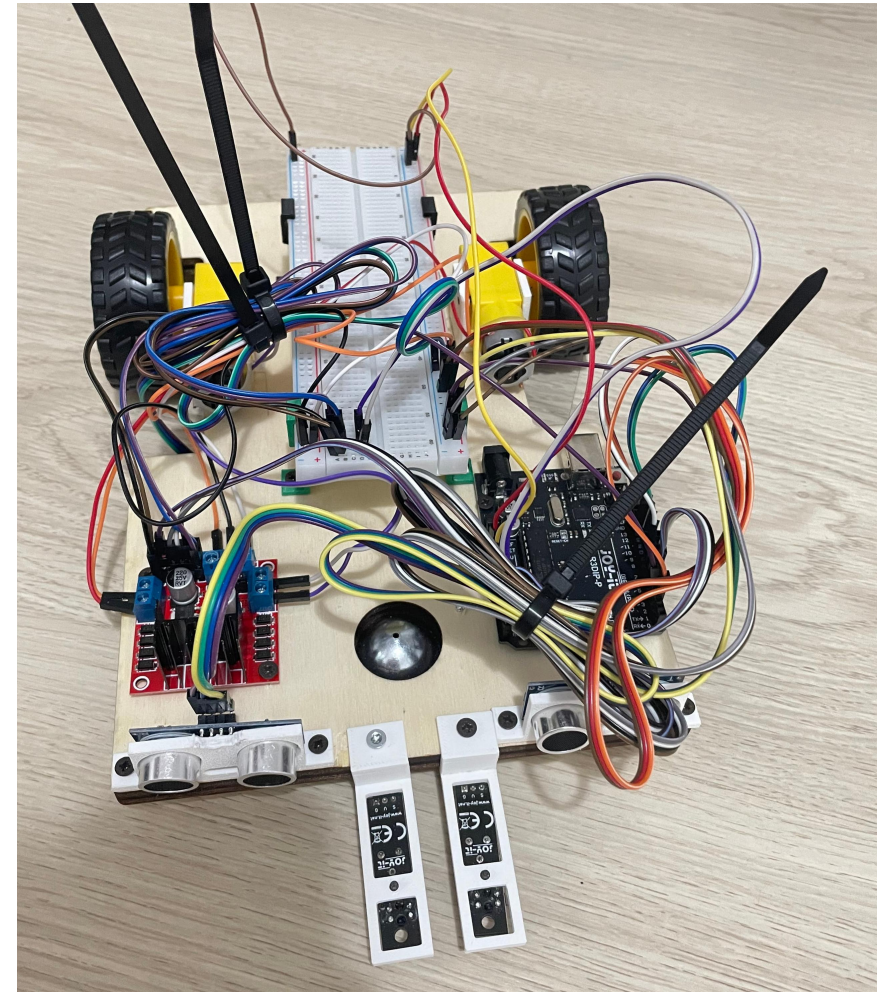
PART 5 Summary



PART 1 3D design



Contributed by team member Yongyu Xu



Contributed by all team members

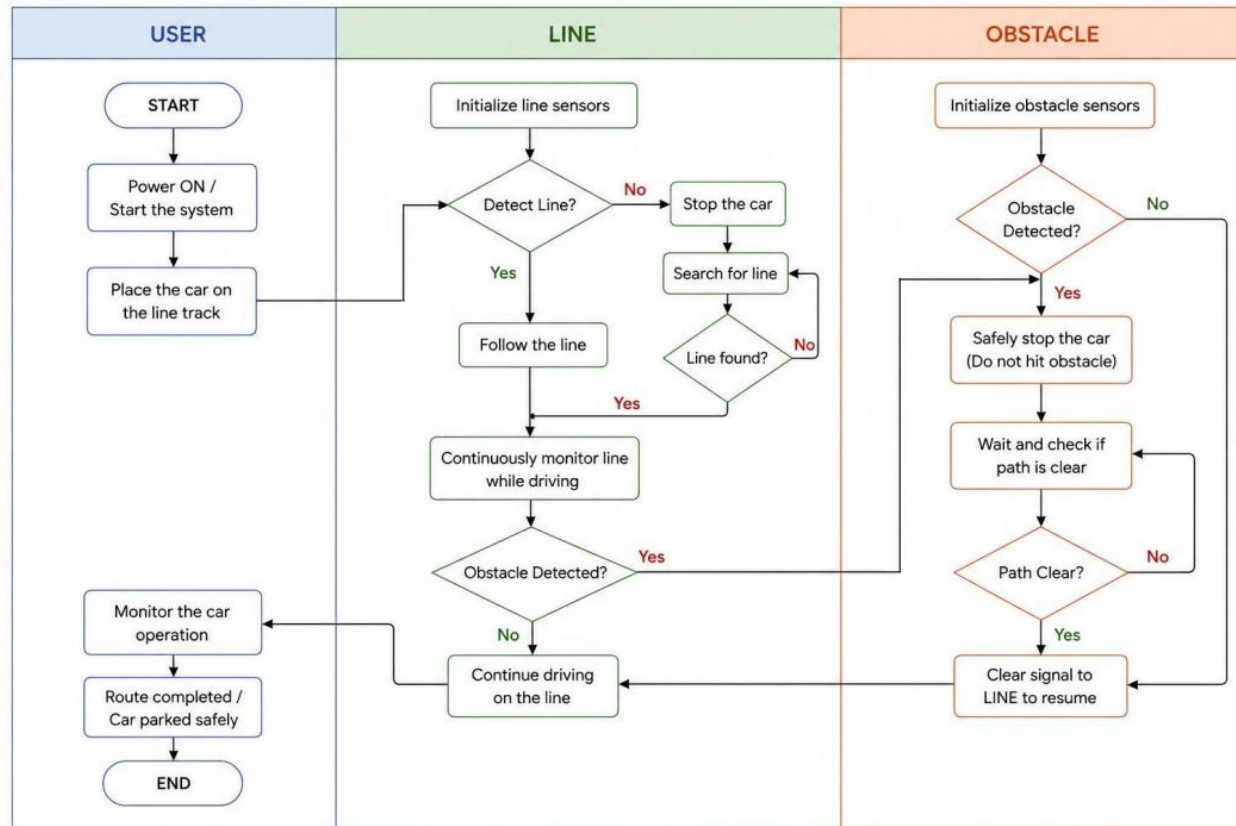


PART 2 Objectives&Approach

Objectives :

Task 1: Develop an autonomous vehicle that can drive autonomously on a given track.

Task 2: Based on line detection, the vehicle can be able to detect obstacles, avoid, search track.



Activity Diagram

Contributed by team member Md Naimur Rahaman Sakib

➤ Obstacle avoidance mode

- Yes-> Stop-> Turn-> Continue tracking
- No-> Line tracking mode

➤ Line following mode

- Read IR sensors-> Move forward/Turn
-> Continue tracking



PART 2 Objectives&Approach

Code Logic

1

Line following

- IR Sensors -> Action
- Both sensors = White -> Move forward
- Left = White, Right = Black -> Turn left
- Left = Black, Right = White -> Turn right
- Both sensors = Black -> Stop or search for the line

2

Obstacle avoidance

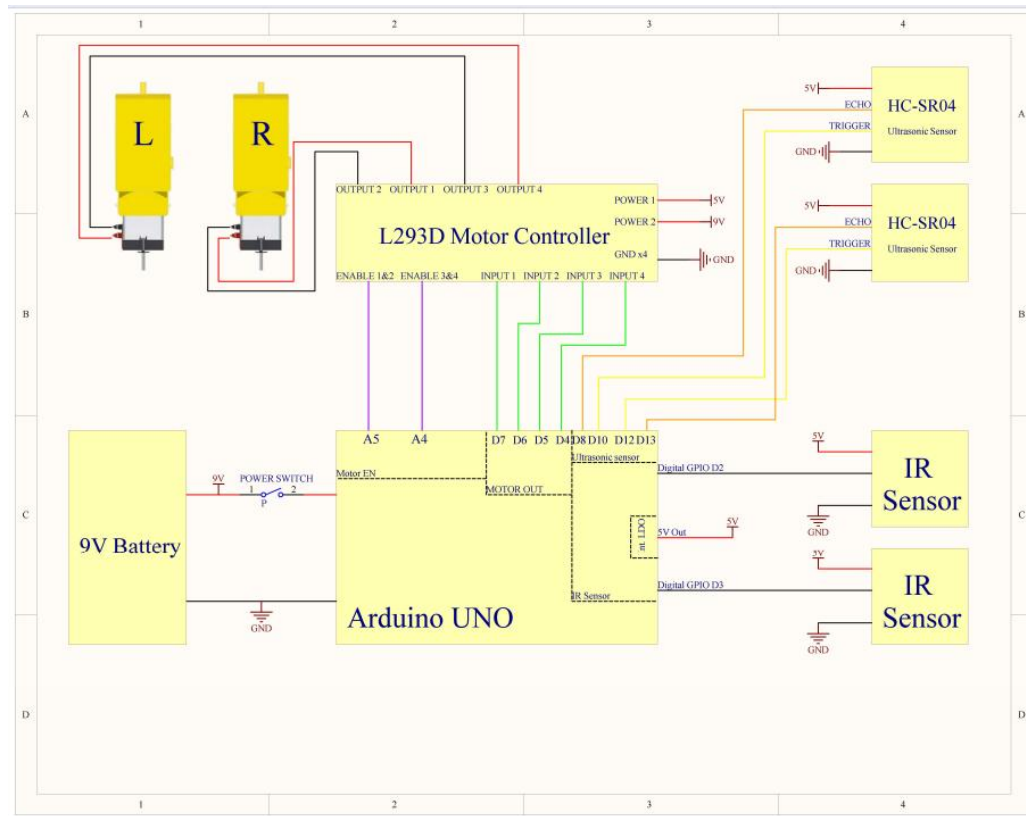
- Ultrasonic Sensors -> Action
- Distance > 20 cm -> Move forward
- Distance ≤ 20 cm -> Stop
- Obstacle detected -> Turn right (or left)
- Path clear -> Resume line tracking

```
void loop() {  
    int rightIR = digitalRead(RIGHT_IR);  
    int leftIR = digitalRead(LEFT_IR);  
  
    Serial.print("Left IR: ");  
    void loop() {  
        updateUltrasonic();  
  
        if (leftDistance < obstacleLimit || rightDistance < obstacleLimit) {  
            stopMotors();  
            delay(1000);  
  
            if (leftDistance > rightDistance) {  
                goAroundLeft();  
            } else {  
                goAroundRight();  
            }  
        } else {  
            followLine();  
        }  
    }  
    forward(); // def fwd  
}
```

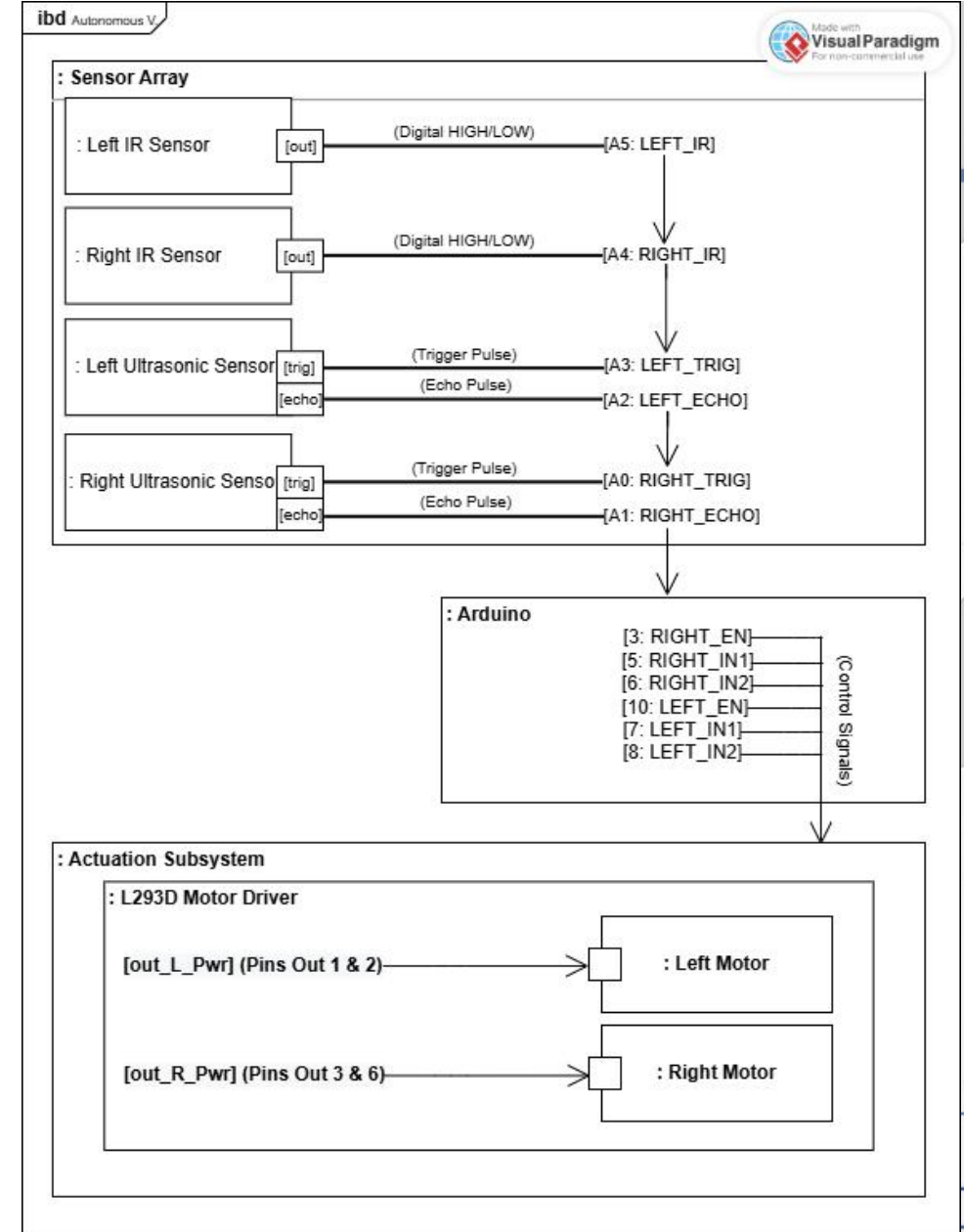


PART 2 Objectives&Approach

Wiring Logic



Physical Schematic
Contributed by team member Baris Koluacik

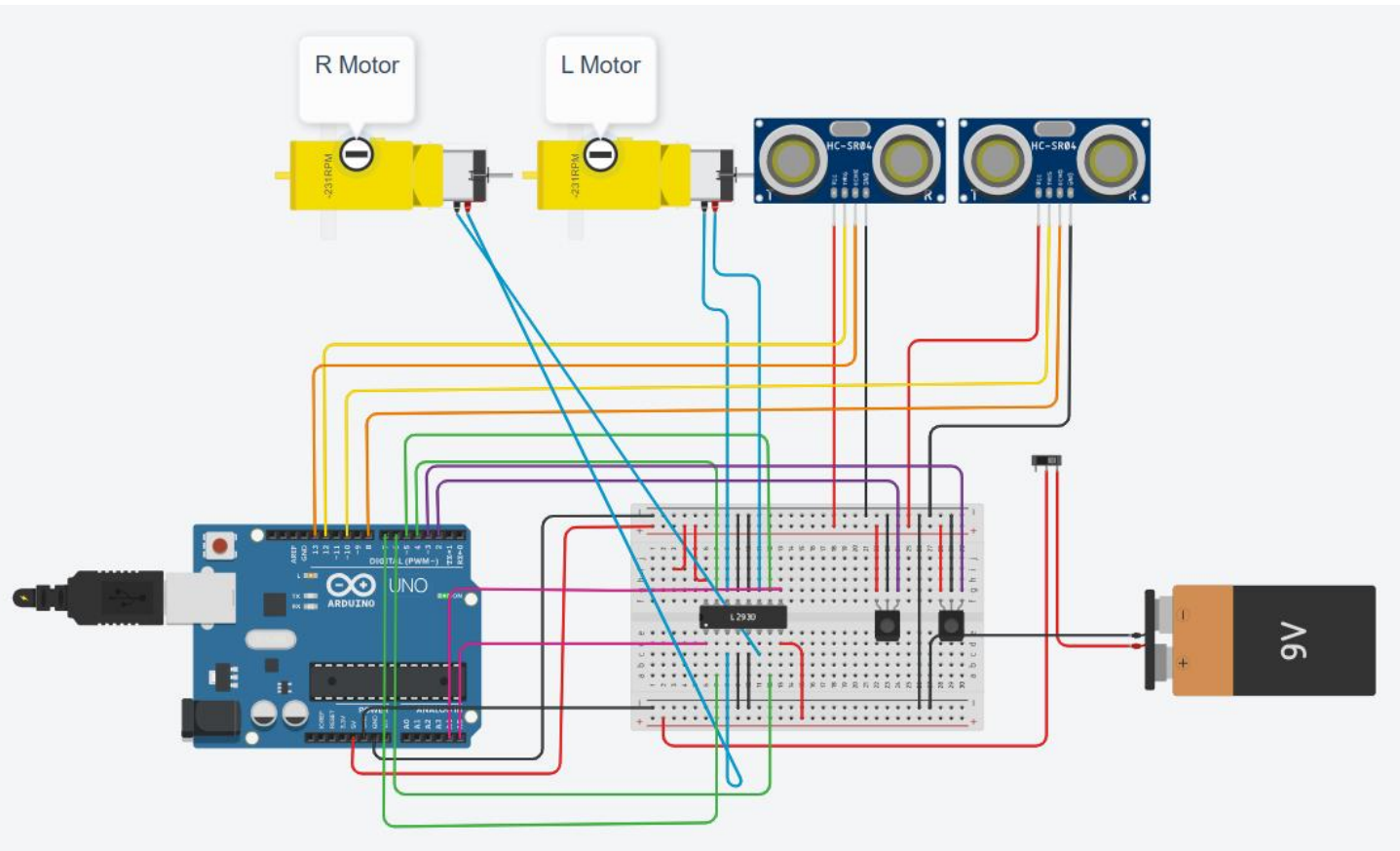


Internal Block Diagram (IBD)
Contributed by team member Md Mahbubur Rahman

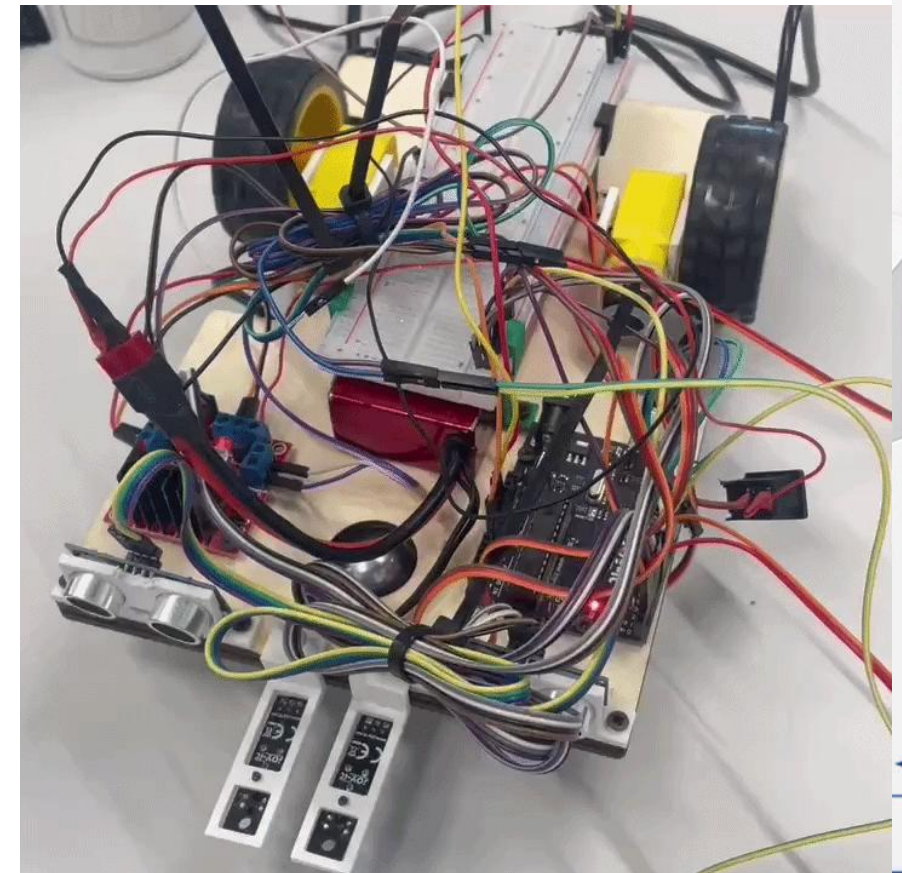


PART 3

Simulation



Simulation on Tinkercad
Contributed by team member Yongyu Xu



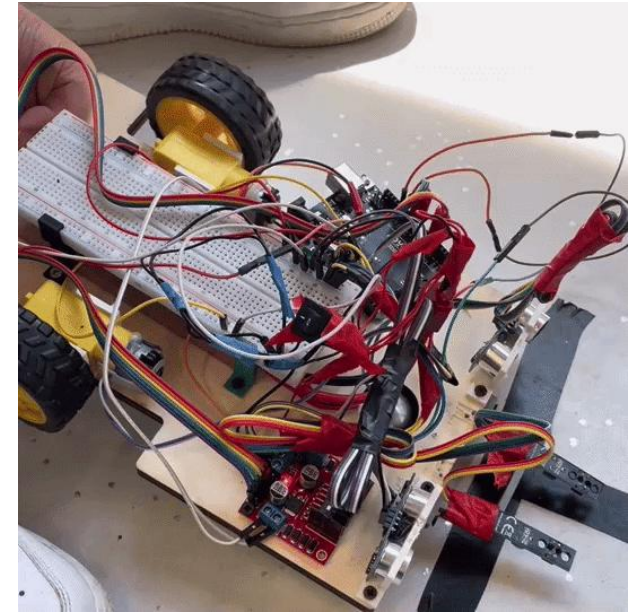
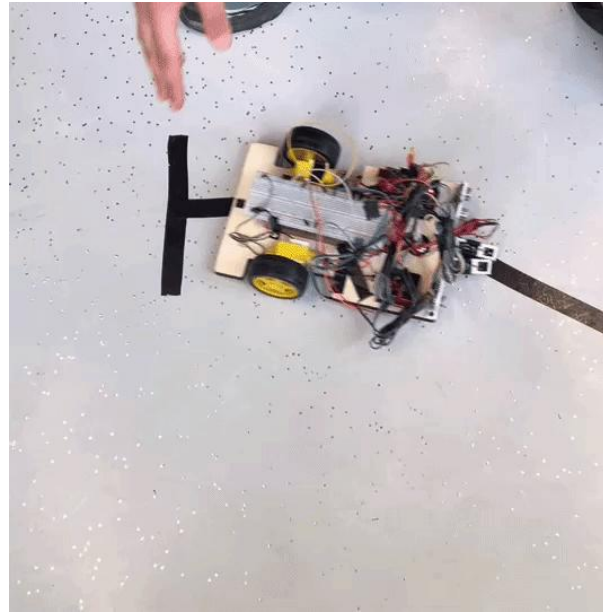
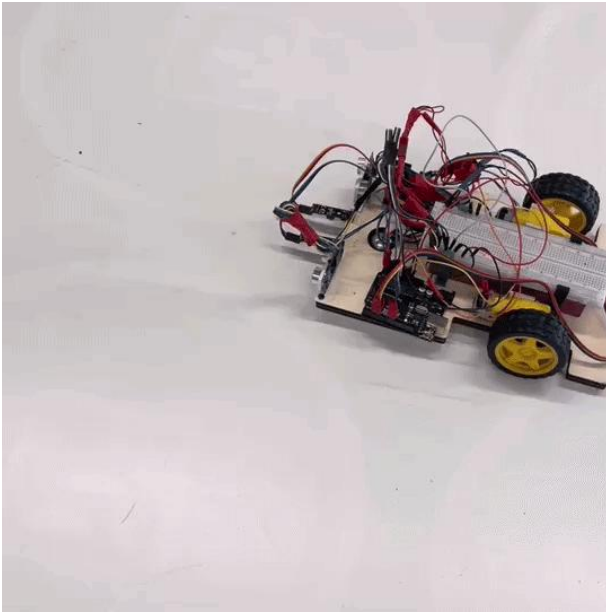
Implementation
Contributed by all team members



PART 4 Issues and Solutions

Issues for Task 1

1. The motors did not move.
2. The vehicle could not move straightly.
3. The vehicle could not follow the line properly. It either stopped after turning or overshot the track after completing a turn.
4. The vehicle worked normally when held above the ground, but failed once it touched the ground.





PART 4 Issues and Solutions

Solutions for Task 1

Wiring

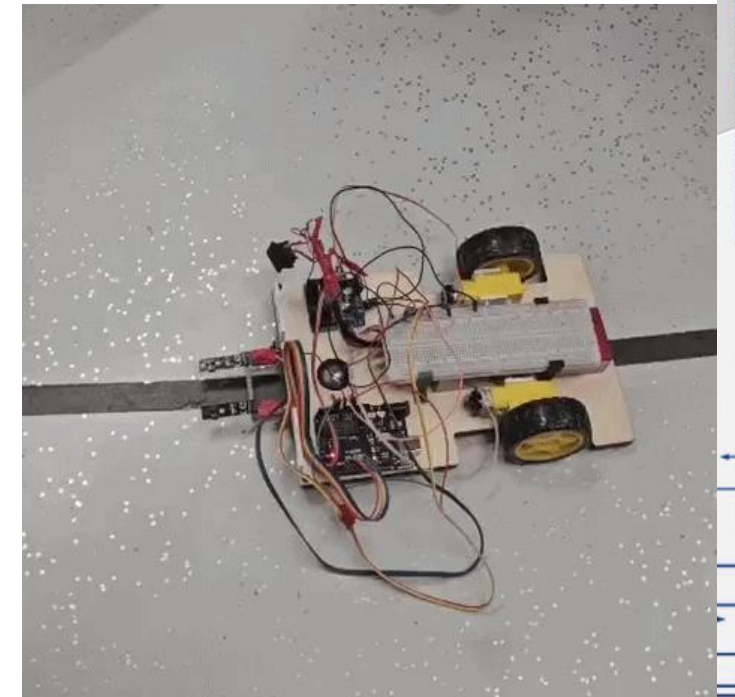
- Incorrect breadboard connections, some components were not connected to the power supply.
- The motor polarity (positive and negative terminals) was connected incorrectly.
- Verified the PWM pin connections.

Code

- Errors in the details

Hardware

- IR sensors -> Good
- Motors -> Good
- Arduino -> Burnt once
- Motor controller -> Broke

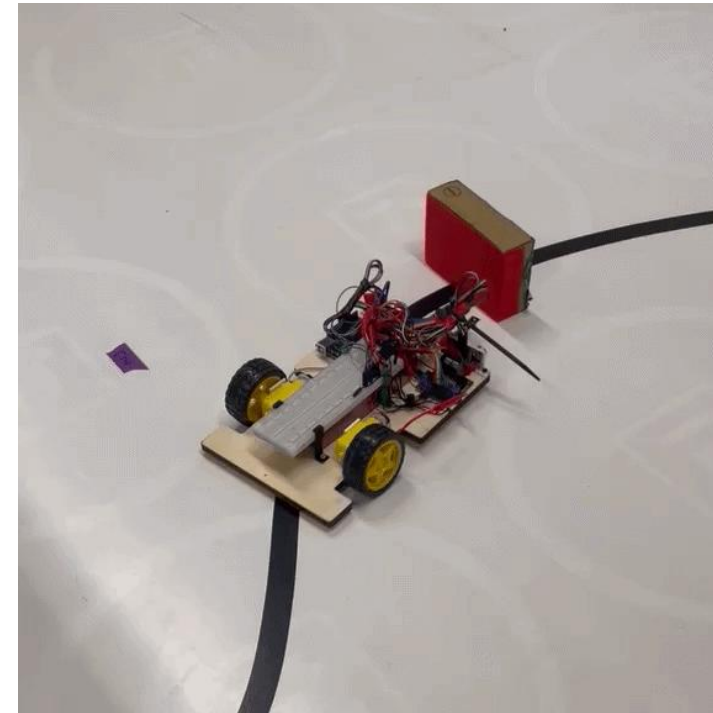
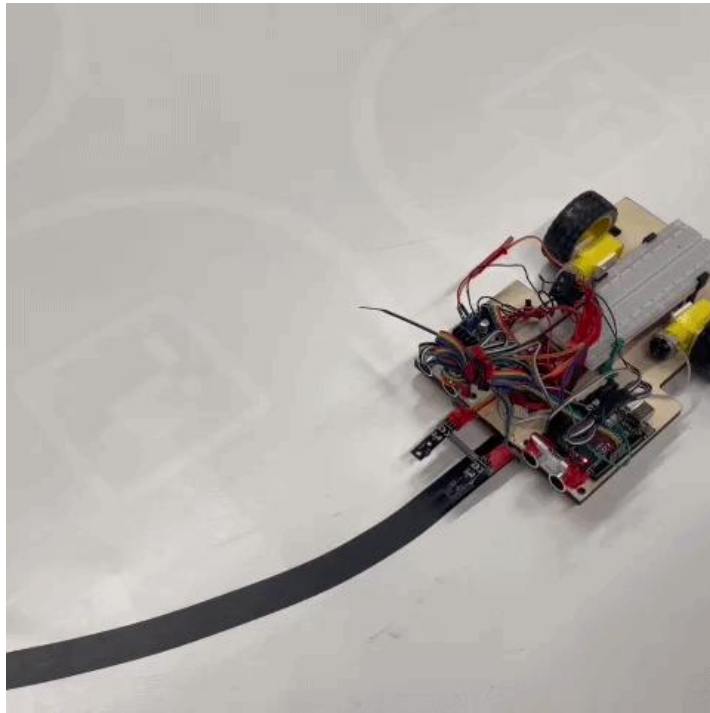




PART 4 Issues and Solutions

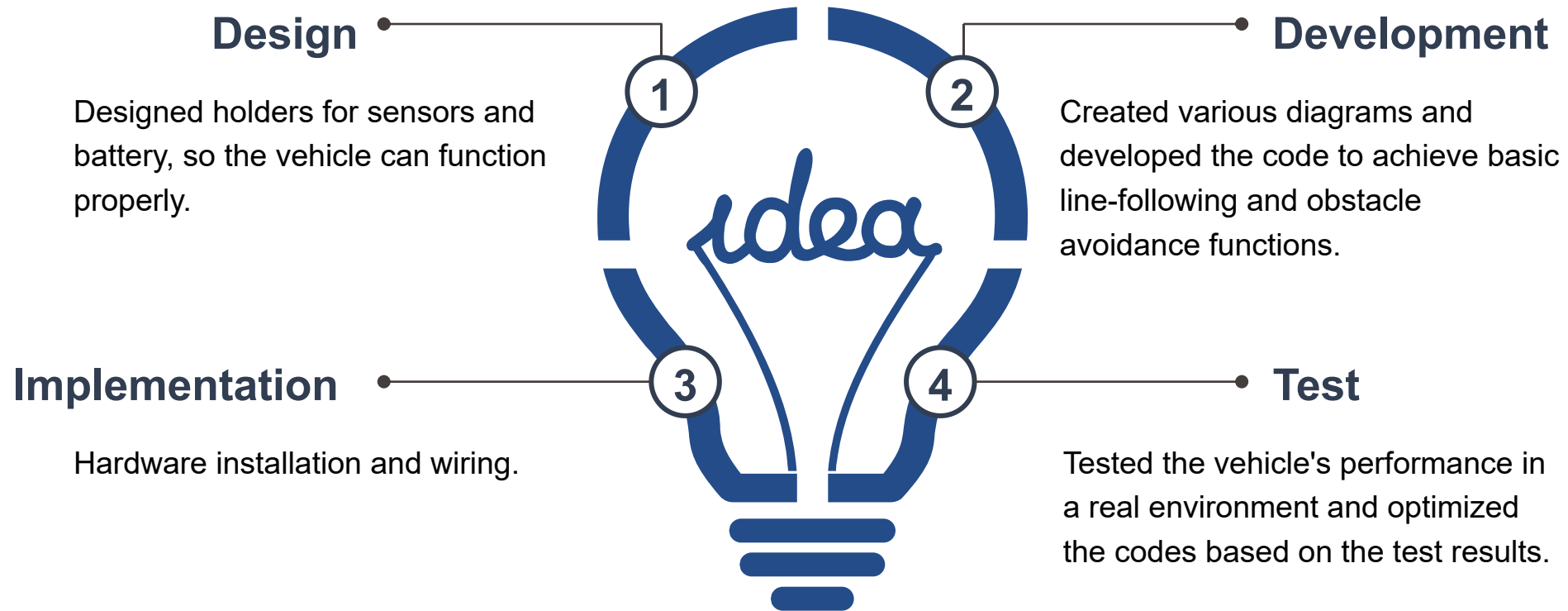
Issues & Solutions for Task 2

1. The vehicle detects the obstacle, then stop.
2. The vehicle detects the obstacle, avoids it, but it couldn't find the line again or follow the line.





PART 5 Summary



Thank you



Q&A

```

60 void loop() {
61     updateUltrasonic();
62
63     if (leftDistance < obstacleLimit || rightDistance < obstacleLimit) {
64         stopMotors();
65         delay(1000);
66
67         if (leftDistance > rightDistance) {
68             goAroundLeft();
69         } else {
70             goAroundRight();
71         }
72     } else {
73         followLine();
74     }
75 }
76
77 void followLine() {
78     int leftIR = digitalRead(LEFT_IR);
79     int rightIR = digitalRead(RIGHT_IR);
80
81     if (leftIR == WHITE && rightIR == WHITE) {
82         forward();
83     }
84     else if (leftIR == WHITE && rightIR == BLACK) {
85         smallTurnLeft();
86     }
87     else if (leftIR == BLACK && rightIR == WHITE) {
88         smallTurnRight();
89     }
90     else {
91         forward();
92     }
93 }
94
95 void goAroundLeft() {
96     turnLeftInPlace();
97     delay(250);
98
99     forward();
100    delay(500);
101
102    turnRightInPlace();
103    delay(250);
104
105    forward();
106    delay(500);
107
108    // Counter direction: search right to go
109    searchLineRight();
110 }
111
112 void goAroundRight() {
113     turnRightInPlace();
114     delay(250);
115
116     forward();
117     delay(500);
118
119     turnLeftInPlace();
120     delay(250);
121
122     forward();
123     delay(500);
124
125     // Counter direction: search left to go
126     searchLineLeft();
127 }
128
129 void searchLineRight() {
130     unsigned long startTime = millis();
131
132     while (millis() - startTime < 6000) {
133         int leftIR = digitalRead(LEFT_IR);
134         int rightIR = digitalRead(RIGHT_IR);
135
136         if (leftIR == BLACK || rightIR == BLACK) {
137             stopMotors();
138             delay(100);
139             return;
140         }
141
142         // Arc right
143         digitalWrite(RIGHT_IN1, HIGH);
144         digitalWrite(RIGHT_IN2, LOW);
145         digitalWrite(LEFT_IN1, HIGH);
146         digitalWrite(LEFT_IN2, LOW);
147
148         analogWrite(RIGHT_EN1, arcSlow);
149         analogWrite(LEFT_EN1, arcFast);
150     }
151
152     stopMotors();
153 }
154
155 void searchLineLeft() {
156     unsigned long startTime = millis();
157
158     while (millis() - startTime < 6000) {
159         int leftIR = digitalRead(LEFT_IR);
160         int rightIR = digitalRead(RIGHT_IR);
161

```

```

162     if (leftIR == BLACK || rightIR == BLACK) {
163         stopMotors();
164         delay(100);
165         return;
166     }
167
168     // Arc left
169     digitalWrite(RIGHT_IN1, HIGH);
170     digitalWrite(RIGHT_IN2, LOW);
171     digitalWrite(LEFT_IN1, HIGH);
172     digitalWrite(LEFT_IN2, LOW);
173
174     analogWrite(RIGHT_EN1, arcFast);
175     analogWrite(LEFT_EN1, arcSlow);
176 }
177
178 stopMotors();
179 }
180
181 void updateUltrasonic() {
182     if (millis() - lastUltrasonicCheck >= ultras
183         lastUltrasonicCheck = millis();
184
185     leftDistance = getDistance(LEFT_TRIG, LEFT
186     rightDistance = getDistance(RIGHT_TRIG, RI
187 }
188 }
189
190 long getDistance(int trigPin, int echoPin) {
191     digitalWrite(trigPin, LOW);
192     delayMicroseconds(2);
193
194     digitalWrite(trigPin, HIGH);
195     delayMicroseconds(10);
196     digitalWrite(trigPin, LOW);
197
198     long duration = pulseIn(echoPin, HIGH, 5000);
199
200     if (duration == 0) {
201         return 999;
202     }
203
204     return duration * 0.034 / 2;
205 }
206
207 void forward() {
208     digitalWrite(RIGHT_IN1, HIGH);
209     digitalWrite(RIGHT_IN2, LOW);
210     digitalWrite(LEFT_IN1, HIGH);
211     digitalWrite(LEFT_IN2, LOW);
212
213     analogWrite(RIGHT_EN1, motorSpeed);
214     analogWrite(LEFT_EN1, motorSpeed);
215 }
216
217 void smallTurnLeft() {
218     digitalWrite(RIGHT_IN1, HIGH);
219     digitalWrite(RIGHT_IN2, LOW);
220     digitalWrite(LEFT_IN1, LOW);
221     digitalWrite(LEFT_IN2, LOW);
222
223     analogWrite(RIGHT_EN1, turnSpeed);
224     analogWrite(LEFT_EN1, 0);
225
226     delay(10);
227
228     analogWrite(RIGHT_EN1, LOW);
229     digitalWrite(RIGHT_IN2, LOW);
230     digitalWrite(LEFT_IN1, HIGH);
231     digitalWrite(LEFT_IN2, LOW);
232
233     analogWrite(RIGHT_EN1, 0);
234     analogWrite(LEFT_EN1, turnSpeed);
235
236     delay(10);
237 }
238
239 void smallTurnRight() {
240
241     void turnLeftInPlace() {
242         digitalWrite(RIGHT_IN1, HIGH);
243         digitalWrite(RIGHT_IN2, LOW);
244         digitalWrite(LEFT_IN1, LOW);
245         digitalWrite(LEFT_IN2, HIGH);
246
247         analogWrite(RIGHT_EN1, turnSpeed);
248         analogWrite(LEFT_EN1, turnSpeed);
249     }
250
251     void turnRightInPlace() {
252         digitalWrite(RIGHT_IN1, LOW);
253         digitalWrite(RIGHT_IN2, HIGH);
254         digitalWrite(LEFT_IN1, HIGH);
255         digitalWrite(LEFT_IN2, LOW);
256
257         analogWrite(RIGHT_EN1, turnSpeed);
258         analogWrite(LEFT_EN1, turnSpeed);
259     }
260

```